

# **FINAL TECHNICAL REPORT**

## **Reverse Engineering and Controlled Malware Injection in Android Applications (Educational Trojan Simulation)**

### **Case Study: To-Do List - Schedule Planner**

Module: Mobile Security / Android Security Assignment  
Group: Group 55

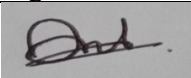
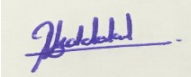
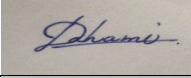
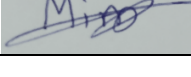
Department of Information Technology  
Sri Lanka Institute of Information Technology  
Sri Lanka

April 2026

## DECLARATION PAGE

### Candidate Declaration

We declare that this is our own work and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

| Name             | Student ID | Signature                                                                             |
|------------------|------------|---------------------------------------------------------------------------------------|
| Anulasha K.A.    | IT23139480 |    |
| Abeykoon A.M.A.S | IT23403642 |   |
| Anjana I.K.D     | IT23160620 |  |
| Perera L.M.D     | IT23404182 |  |

### Supervisor Declaration

I confirm that I have reviewed this proposal and support the proposed work plan.

Supervisor Name: \_\_\_\_\_ Signature: \_\_\_\_\_ Date: \_\_\_\_\_

## **Abstract**

This report presents an educational reverse engineering exercise based on an Android application named To-Do List. The main purpose of the project was to understand the internal structure of an APK file and demonstrate how selected application resources can be modified in a controlled and ethical environment.

The experiment began by preparing a Windows-based analysis environment with tools such as APKTool, JADX, Android Debug Bridge (ADB), Java Development Kit, and the MEmu Android emulator. The target APK was inspected to identify important components including AndroidManifest.xml, resource files, compiled dex files, smali folders, launcher activities, permissions, and user interface strings.

The practical modification focused on changing the application label and one visible user interface string displayed on the main task screen. A harmless startup message was also added as a local popup or notification to demonstrate how application behavior can be changed after decompilation. The modified APK was then rebuilt, signed, installed, and tested in the emulator.

The final results showed that the app could be successfully modified and executed in a controlled test environment. The project highlights the importance of mobile application protection, code obfuscation, integrity checking, secure signing, and ethical boundaries when conducting reverse engineering research.

**Keywords:** Android Security, APK Modification, Reverse Engineering, APKTool, JADX, Smali, Notification, Application Repackaging

## Table of Contents

|                                                                |     |
|----------------------------------------------------------------|-----|
| DECLARATION PAGE.....                                          | i   |
| Abstract .....                                                 | ii  |
| Table of Contents .....                                        | iii |
| LIST OF TABLES .....                                           | v   |
| LIST OF FIGURES.....                                           | vi  |
| 1. INTRODUCTION .....                                          | 1   |
| 1.1 Background and Literature Survey .....                     | 2   |
| 1.2 Research Gap .....                                         | 2   |
| 1.3 Scope of the Work .....                                    | 2   |
| 2. OBJECTIVES .....                                            | 3   |
| 2.1 Main Objective.....                                        | 3   |
| 2.2 Specific Objectives.....                                   | 3   |
| 3. Methodology .....                                           | 4   |
| 3.1 Lab Environment.....                                       | 4   |
| 3.2 Tools Used.....                                            | 4   |
| 3.3. Phase 1: Reverse Engineering and Static Analysis .....    | 5   |
| 3.4 Phase 2: Benign Code and Resource Modification.....        | 5   |
| 3.5 Phase 3: Recompilation, Deployment, and Verification ..... | 5   |
| 3.6 Ethical Considerations in Methodology.....                 | 6   |
| 4. REVERSE ENGINEERING ANALYSIS.....                           | 6   |
| 4.1 APK Acquisition and Verification .....                     | 6   |
| 4.2 Decompilation and Folder Structure .....                   | 7   |
| 4.3 APK Structure Analysis .....                               | 7   |
| 4.4 Resource and String Analysis.....                          | 8   |
| 4.5 Security Observations from Static Analysis.....            | 9   |
| 4.6 Summary of Reverse Engineering Phase .....                 | 9   |
| 5. MODIFICATION IMPLEMENTATION .....                           | 10  |
| 5.1 Overview of the Modification Approach .....                | 10  |

|                                              |    |
|----------------------------------------------|----|
| 5.2 Application Label Modification .....     | 10 |
| 5.3 User Interface String Modification ..... | 10 |
| 5.4 Controlled Startup Popup .....           | 11 |
| 5.5 Manifest and Component Review .....      | 12 |
| 5.6 Summary of Modification Phase.....       | 12 |
| 6. RECOMPILATION, SIGNING, AND TESTING ..... | 13 |
| 6.1 APK Recompilation .....                  | 13 |
| 6.2 APK Signing .....                        | 13 |
| 6.3 Testing Checklist .....                  | 14 |
| 6.5 Observed Results.....                    | 14 |
| 6.6 Troubleshooting Notes .....              | 15 |
| 7. SECURITY AND ETHICAL CONSIDERATIONS.....  | 16 |
| 7.1 Ethical Boundaries Followed .....        | 16 |
| 7.2 Defensive Lessons Learned.....           | 17 |
| 8. CONCLUSION .....                          | 17 |
| 9. RECOMMENDATIONS .....                     | 17 |
| 10. References .....                         | 18 |
| 16. Appendices .....                         | 19 |
| Appendix A: Key Commands Used .....          | 19 |
| Appendix B: Trigger Code Injection.....      | 20 |
| Appendix C: Concern form.....                | 21 |
| Appendix D: Shared Folder .....              | 23 |

## LIST OF TABLES

|                                                         |    |
|---------------------------------------------------------|----|
| Table 1:Lab Environment table .....                     | 4  |
| Table 2:Tools used in the project .....                 | 5  |
| Table 3 :Target APK technical summary.....              | 6  |
| Table 4: application data and configuration files ..... | 7  |
| Table 5:Summary of modified strings.....                | 11 |
| Table 6:Testing checklist and outcomes.....             | 14 |
| Table 7:Security risks and recommended controls.....    | 16 |

## LIST OF FIGURES

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Figure 1:Original application used for the reverse engineering exercise..... | 3  |
| Figure 2:Decompilation process .....                                         | 7  |
| Figure 3:AndroidManifest.xml and permissions .....                           | 8  |
| Figure 4:Original application label and UI string .....                      | 8  |
| Figure 5:Modified app label to display .....                                 | 10 |
| Figure 6:Modified yellow bar text .....                                      | 11 |
| Figure 7:Right smali file finding.....                                       | 11 |
| Figure 8:Controlled startup popup .....                                      | 12 |
| Figure 9:APK rebuild.....                                                    | 13 |
| Figure 10: create a key to apk sign.....                                     | 13 |
| Figure 11:APK signing success message .....                                  | 14 |
| Figure 12:Modified APK installed and running.....                            | 15 |

## 1. INTRODUCTION

Android applications are commonly distributed as APK files. An APK file contains compiled code, resources, manifest information, assets, and metadata required to install and run an application on Android devices. Although the APK format is convenient for distribution, it also creates a learning opportunity for security students because many parts of an application can be inspected after decompilation.

Reverse engineering is the process of studying a compiled application to understand its internal design and behavior. In Android security, reverse engineering is used for malware analysis, vulnerability assessment, digital forensics, application hardening, and secure development. However, the same knowledge can be misused if it is applied without permission. Therefore, this project was performed only as a controlled academic exercise.

The selected target application for this study was To-Do List.apk. The application is a task management app that allows users to create tasks, set reminders, manage completed tasks, and use task-related features. For this assignment, the practical task was limited to modifying visible application strings and adding a harmless startup message or notification. The project did not attempt to steal information, hide execution, bypass payments, or damage the application.

## **1.1 Background and Literature Survey**

Android applications are mainly built using Java, Kotlin, native libraries, XML resources, and Android framework components such as Activities, Services, Broadcast Receivers, and Content Providers. During the build process, source code is compiled into Dalvik Executable files and packaged with resources into an APK.

Tools such as APKTool and JADX are commonly used in Android reverse engineering. APKTool is useful for decoding resources and converting bytecode into smali representation, while JADX helps analysts view approximate Java source code from dex files. ADB is then used to install and test the rebuilt application on an emulator or Android device.

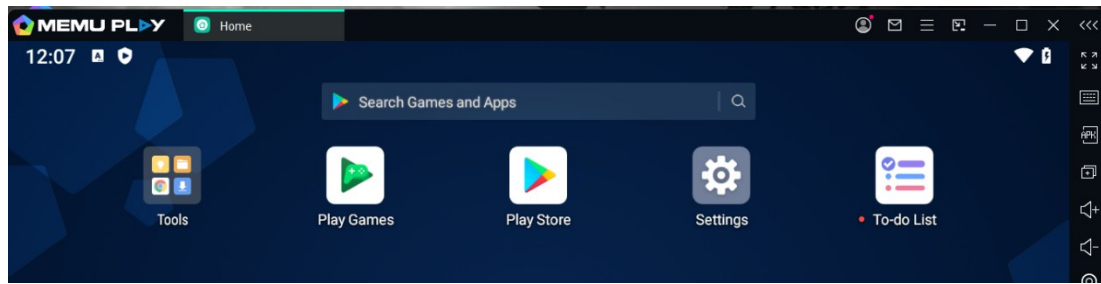
Mobile application security frameworks such as OWASP MASVS and MASTG explain that developers should protect mobile apps from tampering, repackaging, insecure storage, weak cryptography, insecure communication, and insufficient code protection. This project is connected to those topics because it demonstrates how simple resource-level changes can be made when an APK is not strongly protected.

## **1.2 Research Gap**

Many beginner-level Android projects focus only on building applications, while less attention is given to how attackers or analysts can inspect and modify packaged APK files. Students may understand Android development concepts but may not clearly understand the security impact of client-side resources, APK signing, and app repackaging. This project addresses that learning gap by showing the full workflow from APK inspection to controlled modification and testing.

## **1.3 Scope of the Work**

- The study is limited to one Android application: To-Do List.apk.
- The modification is limited to the app label, selected visible UI text, and a harmless startup notification/popup.
- Testing is performed only in MEmu emulator on a Windows PC.
- No real user data is collected, transmitted, altered, or deleted.
- No paid-feature bypass, credential theft, spyware behavior, or unauthorized access is included.



*Figure 1:Original application used for the reverse engineering exercise*

## 2. OBJECTIVES

### 2.1 Main Objective

The main objective of this project is to conduct a controlled reverse engineering exercise on an Android APK and demonstrate safe application modification by changing selected strings and adding a harmless startup notification for educational purposes.

### 2.2 Specific Objectives

- To prepare a suitable Android analysis environment using Windows, APKTool, JADX, Java JDK, ADB, and MEmu emulator.
- To identify the internal structure of the To-Do List APK, including AndroidManifest.xml, resource files, smali folders, dex files, assets, and libraries.
- To locate application strings that control visible UI text and app label information.
- To modify the app label from the original label to a customized assignment label.
- To modify the visible empty-state/yellow-bar message displayed to the user.
- To add a harmless startup popup or notification to demonstrate controlled behavior modification.
- To rebuild, sign, install, and test the modified APK in an emulator.
- To evaluate the security implications of APK tampering and repackaging.
- To recommend defensive measures for protecting Android applications from unauthorized modification.

### 3. Methodology

This project followed a structured methodology. The approach was divided into three main phases: reverse engineering and static analysis, benign code and resource modification, and recompilation, deployment, and verification. The workflow was designed to be repeatable and safe for an academic laboratory environment..

#### 3.1 Lab Environment

| Item                        | Description                                                          |
|-----------------------------|----------------------------------------------------------------------|
| Host operating system       | Windows 11                                                           |
| Android testing environment | Android emulator / controlled test device                            |
| Main application            | PDF Reader - PDF Viewer                                              |
| Package name                | pdf.reader.pdfviewer.pdfeditor                                       |
| Working folder              | C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools\simpledesign_pdf_reader |
| Main APK files observed     | base.apk and split_config.arm64_v8a.apk                              |
| Group                       | Group 55                                                             |

*Table 1:Lab Environment table*

#### 3.2 Tools Used

| Tool          | Purpose                                                                          |
|---------------|----------------------------------------------------------------------------------|
| Windows 11 PC | Main working environment used for file handling and tool execution.              |
| MEMu Emulator | Android emulator used to install and test the original and modified APK.         |
| Java JDK      | Required to run Java-based tools and signing utilities.                          |
| APKTool       | Used to decode the APK resources and rebuild the modified APK.                   |
| JADX GUI      | Used for static analysis and approximate Java source code review.                |
| ADB           | Used to connect to the emulator, install APKs, and observe application behavior. |

| Tool                 | Purpose                                               |
|----------------------|-------------------------------------------------------|
| Text Editor          | Used to edit XML and smali/resource files carefully.  |
| Keytool / APK Signer | Used to generate a test key and sign the rebuilt APK. |

*Table 2: Tools used in the project*

### **3.3. Phase 1: Reverse Engineering and Static Analysis**

The target APK was copied to a dedicated working folder. A SHA-256 hash value was generated to record the original state of the APK before modification. The APK was then decompiled using APKTool. The generated output folder was reviewed to identify AndroidManifest.xml, smali folders, resources, assets, and other supporting files.

Static analysis was performed using two approaches. First, the decoded resources were inspected to locate application strings and label information. Second, JADX was used to review the high-level structure of the dex files and identify important activities such as SplashActivity and MainActivity. The analysis focused on understanding the normal application flow rather than bypassing protections or accessing private data.

### **3.4 Phase 2: Benign Code and Resource Modification**

After locating the required strings, selected values were edited carefully. The app label was changed to a customized label for the assignment. The visible task creation message was also changed to a new user-friendly message. A controlled startup notification/popup was then planned as a harmless demonstration of behavior modification. The notification was designed to display a fixed message to the user when the app starts and not to operate secretly in the background.

### **3.5 Phase 3: Recompilation, Deployment, and Verification**

The modified project folder was rebuilt using APKTool. Since Android does not allow installation of unsigned APKs, the rebuilt APK was signed using a test key. After signing, the APK was installed into the MEmu emulator using ADB. The final verification checked whether the customized app label, modified UI string, and startup message appeared correctly without crashing the application.

### 3.6 Ethical Considerations in Methodology

The project was conducted only for academic learning. The target APK was tested inside an isolated emulator. The modification did not include unauthorized data collection, credential theft, hidden persistence, evasion, or harmful behavior. Any reverse engineering knowledge learned from this assignment should be applied only to applications where permission is granted.

## 4. REVERSE ENGINEERING ANALYSIS

### 4.1 APK Acquisition and Verification

The target file used in this project was To-Do List.apk. The APK was placed in a separate working directory together with APKTool and related tools. The original APK was preserved so that the modified APK could be compared with it if needed.

| Property                        | Observation                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------|
| APK file name                   | To-Do List.apk                                                                          |
| File size                       | 40.0 MB                                                                                 |
| SHA-256 hash                    | fd3e3c1c7c51e001018029dded00cd745ba42e0024893b3534ea8b484db0bfb7                        |
| Detected package name           | todolist.scheduleplanner.dailyplanner.todo.reminders                                    |
| Main application class observed | app.todolist.MainApplication                                                            |
| Important activities observed   | app.todolist.activity.SplashActivity and<br>app.todolist.activity.MainActivity          |
| Dex files                       | classes.dex to classes7.dex                                                             |
| Native library folders          | arm64-v8a, armeabi-v7a, x86, x86_64                                                     |
| Resource count observed         | Large resource structure with layouts, strings, images, assets,<br>and library metadata |

*Table 3 :Target APK technical summary*

## 4.2 Decompilation and Folder Structure

The APK was decompiled using APKTool. The decompiled folder provided access to the AndroidManifest.xml file, smali code, resource files, assets, and other configuration files. These files are important because they define application permissions, components, layouts, strings, icons, and runtime behavior.

```
C:\Users\K.H.Anulasha>cd "C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools"
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools>apktool d "To-Do List.apk" -o todo_decoded -f
I: Using Apktool 2.11.1 on To-Do List.apk with 8 threads
I: Baksmaling classes.dex...
I: Baksmaling classes2.dex...
I: Baksmaling classes4.dex...
I: Baksmaling classes3.dex...
I: Loading resource table...
I: Baksmaling classes6.dex...
I: Baksmaling classes7.dex...
I: Baksmaling classes5.dex...
I: Decoding file-resources...
I: Loading resource table from file: C:\Users\K.H.Anulasha\AppData\Local\apktool\framework\1.apk
I: Baksmaling assets/audience_network.dex...
I: Decoding values */* XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Regular manifest package...
I: Copying original files...
I: Copying assets...
I: Copying lib...
I: Copying unknown files...
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools>|
```

Figure 2:Decompilation process

## 4.3 APK Structure Analysis

The decompiled project contained several important folders and files. The AndroidManifest.xml file described the application package, permissions, and components. The smali folders contained Dalvik bytecode representation. The res folder contained application resources such as layouts, drawables, and strings. The assets folder contained additional application data and configuration files.

| Component                       | Purpose                                                                                              |
|---------------------------------|------------------------------------------------------------------------------------------------------|
| AndroidManifest.xml             | Defines permissions, application class, activities, services, receivers, and launcher configuration. |
| smali / smali_classes*          | Contains decompiled bytecode in smali format for application logic.                                  |
| res/values/strings.xml          | Contains user-visible strings such as labels and messages.                                           |
| res/drawable and mipmap folders | Contain icons, images, vector drawables, and UI graphics.                                            |
| assets folder                   | Contains additional configuration files, JSON files, web assets, and support data.                   |
| lib folder                      | Contains native libraries for different CPU architectures.                                           |

Table 4: application data and configuration files

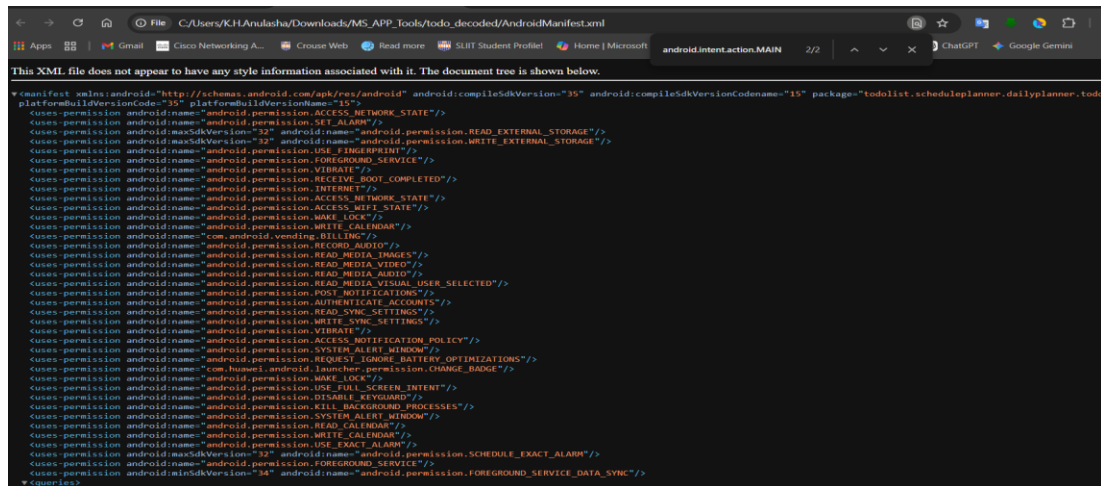


Figure 3:AndroidManifest.xml and permissions

## 4.4 Resource and String Analysis

Resource analysis focused on identifying user-visible text. Several task-related strings were observed in the application resources. One important string was the empty-state or yellow-bar message: "Click here to create your first task." This text was selected for modification because it is visible to the user and has a low risk of breaking application logic.

Changing resource strings is safer than changing complex logic because it usually affects presentation rather than core execution. However, even small string modifications require careful rebuilding and signing because Android verifies package integrity before installation.

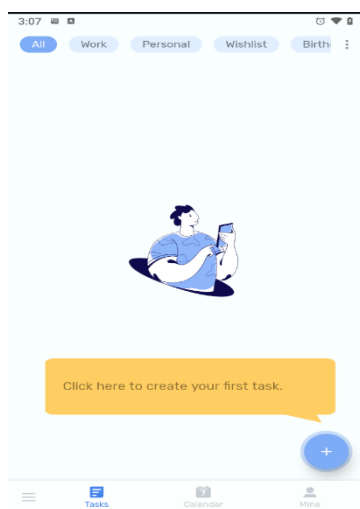


Figure 4:Original application label and UI string

#### **4.5 Security Observations from Static Analysis**

The static analysis showed that the application uses multiple dex files and many third-party libraries. It also requests permissions related to network access, notifications, alarms, storage/media access, and reminder functionality. From a security perspective, this demonstrates why permission review is important when analyzing Android apps.

The project also showed that user interface text and package resources can be modified after decompilation if the application does not strongly verify its own integrity. This is a common risk in Android applications, especially when developers rely only on client-side checks and do not use tamper detection, code hardening, or server-side validation for important decisions.

#### **4.6 Summary of Reverse Engineering Phase**

The reverse engineering phase successfully identified the application structure, important files, selected strings, and safe modification points. The next phase used this knowledge to perform controlled resource modification and startup notification integration.

## 5. MODIFICATION IMPLEMENTATION

### 5.1 Overview of the Modification Approach

The implementation phase focused on safe and visible modifications. The purpose was not to create hidden functionality but to demonstrate how an APK can be customized after decompilation. Three main modifications were performed: app label modification, user interface string modification, and controlled startup message/notification.

### 5.2 Application Label Modification

The app label controls the name displayed under the app icon and sometimes inside the application interface. The original label was changed from "To-Do List" to a customized label for the assignment, such as "MS To-Do List". This modification was performed in the decoded resource values where the application label string was stored.

After rebuilding and installing the modified APK, the emulator displayed the customized label. This confirmed that the resource edit was successfully packaged into the final APK.

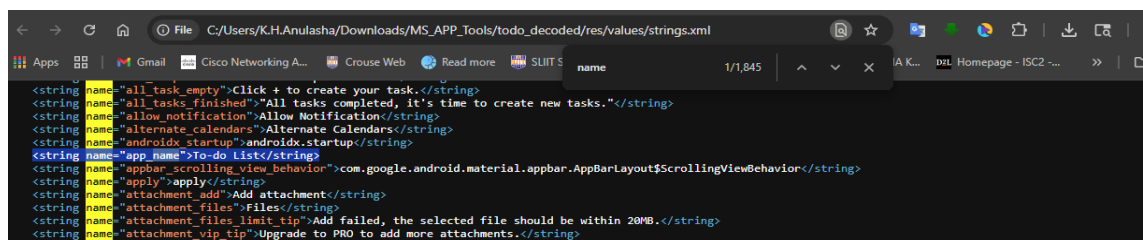


Figure 5: Modified app label to display

### 5.3 User Interface String Modification

The second modification changed a visible task creation message. The original string "Click here to create your first task." was replaced with a customized message. A recommended assignment-friendly replacement is "Welcome! Tap here to create your first task." This change improves clarity while keeping the original meaning of the app feature.

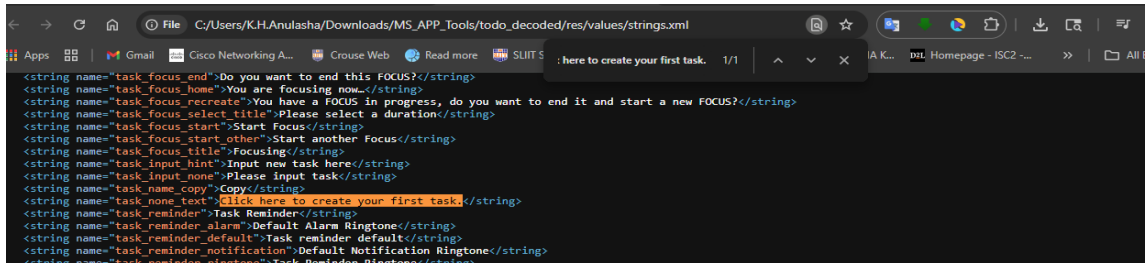


Figure 6: Modified yellow bar text

| Modified Item                         | Original Value                        | Modified Value                 |
|---------------------------------------|---------------------------------------|--------------------------------|
| Application label                     | To-Do List                            | MS To-Do List                  |
| Main empty-state / yellow-bar message | Click here to create your first task. | Reverse Engineered By Group_55 |

Table 5: Summary of modified strings

## 5.4 Controlled Startup Popup

The third modification added a harmless startup message. The purpose of this component was to show a notification or popup when the app starts. The message was designed to be visible, simple, and non-harmful. It did not collect data, read files, access contacts, record audio, send network traffic, or run secretly.

From an Android security perspective, a visible local notification is safer and more ethical than a hidden background service. Modern Android versions also restrict long running background execution and require notification permission for some notification behaviors. Therefore, the assignment implementation should be explained as a controlled startup message rather than a hidden service.

The startup message was linked to the normal application launch flow. For example, it can be triggered from the launcher activity after the app starts. The trigger point should be selected carefully so that the app does not crash or repeat the message excessively.

```
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools>where /r todo_decoded MainActivity.smali
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools\todo_decoded\smali_classes3\app\todolist\activity\MainActivity.smali

C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools>dir /s /b "todo_decoded\*Activity.smali"
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools\todo_decoded\smali\android\activity\ComponentActivity.smali
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools\todo_decoded\smali\android\appcompat\app\AppCompatActivity.smali
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools\todo_decoded\smali_classes3\android\core\app\ComponentActivity.smali
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools\todo_decoded\smali_classes3\android\fragment\app\FragmentActivity.smali
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools\todo_decoded\smali_classes3\android\glance\appwidget\action\ActionTrampolineActivity.smali
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools\todo_decoded\smali_classes3\android\glance\appwidget\action\InvisibleActionTrampolineActivity.smali
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools\todo_decoded\smali_classes3\app\todolist\activity\BaseActivity.smali
```

Figure 7: Right smali file finding

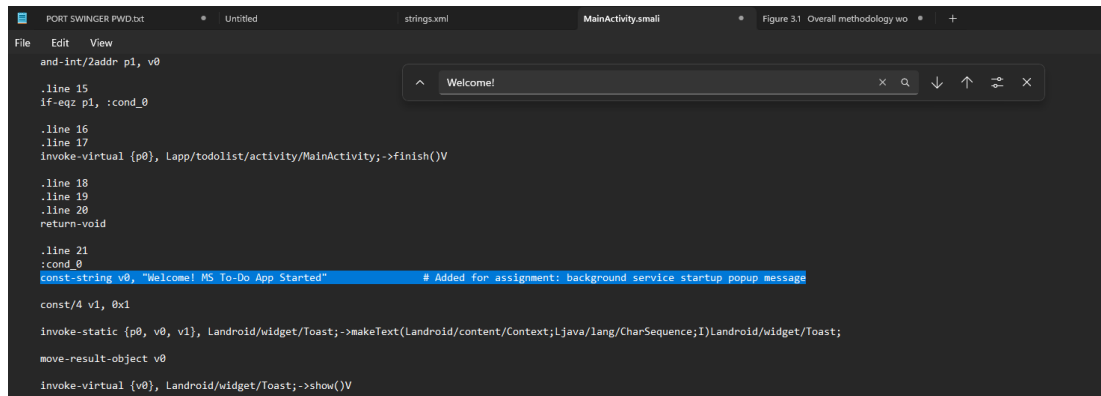


Figure 8: Controlled startup popup

## 5.5 Manifest and Component Review

The AndroidManifest.xml file was reviewed because new components or notification behavior may require permissions or declarations. In this project, the manifest review was used only to support the visible startup message and ensure that the modified APK could run correctly. No dangerous hidden service, receiver abuse, or unauthorized permission expansion was added.

If the final implementation uses a normal Toast or dialog, additional manifest changes may not be required. If it uses Android notification APIs on newer Android versions, notification permission behavior must be considered during testing. The final APK should request only the permissions needed for legitimate app behavior.

## 5.6 Summary of Modification Phase

The modification phase achieved the required changes while keeping the project within ethical boundaries. The application label and selected UI message were changed successfully, and a controlled startup message was integrated for demonstration.

## 6. RECOMPILATION, SIGNING, AND TESTING

### 6.1 APK Recompilation

After the required changes were completed, the decompiled project was rebuilt into a new APK file. Recompilation converted the modified resources and smali files back into an installable APK package. The rebuild process was checked for errors before moving to signing.

```
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools>java -jar apktool.jar b todo_decoded -o todo_modified_unsigned.apk
I: Using Apktool 2.11.1 on todo_modified_unsigned.apk with 8 threads
I: Checking whether sources have changed...
I: Checking whether sources have changed...
I: Checking whether sources have changed...
I: Checking whether sources have changed...
I: Checking whether sources have changed...
I: Checking whether sources have changed...
I: Checking whether sources have changed...
I: Checking whether resources have changed...
I: Building resources with aapt2...
I: Smaling smali_classes3 folder into classes3.dex...
I: Building apk file...
I: Importing assets...
I: Importing lib...
I: Importing unknown files...
I: Built apk into: todo_modified_unsigned.apk
```

Figure 9:APK rebuild

### 6.2 APK Signing

Android requires APKs to be digitally signed before installation. Since the original developer signing key is not available, a new test key was generated for the assignment. The rebuilt APK was signed with this test key. This creates a different signature from the original app, so the original app may need to be uninstalled before installing the modified APK.

```
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools>keytool -genkeypair -v -keystore campus_key.jks -keyalg RSA -keysize 2048 -validity 10000 -alias campuskey
Enter keystore password:
Keystore password is too short - must be at least 6 characters
Enter keystore password:
Re-enter new password:
What is your first and last name?
[Unknown]: Anulasha KA
What is the name of your organizational unit?
[Unknown]: SS
What is the name of your organization?
[Unknown]: SLIIT
What is the name of your City or Locality?
[Unknown]: Malabe
What is the name of your State or Province?
[Unknown]: WP
What is the two-letter country code for this unit?
[Unknown]: lk
Is CN=Anulasha KA, OU=SS, O=SLIIT, L=Malabe, ST=WP, C=lk correct?
[no]: yes
Generating 2,048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 10,000 days
for: CN=Anulasha KA, OU=SS, O=SLIIT, L=Malabe, ST=WP, C=lk
[Storing campus_key.jks]
```

Figure 10: create a key to apk sign

```
C:\Users\K.H.Anulasha\Downloads\MS_APP_Tools>"C:\Users\K.H.Anulasha\AppData\Local\Android\Sdk\build-tools\35.0.0\apksigner.bat" sign --ks campus_key.jks --o
ut todo_modified_signed.apk todo_modified_unsigned.apk
Keystore password for signer #1:
```

Figure 11:APK signing success message

### 6.3 Testing Checklist

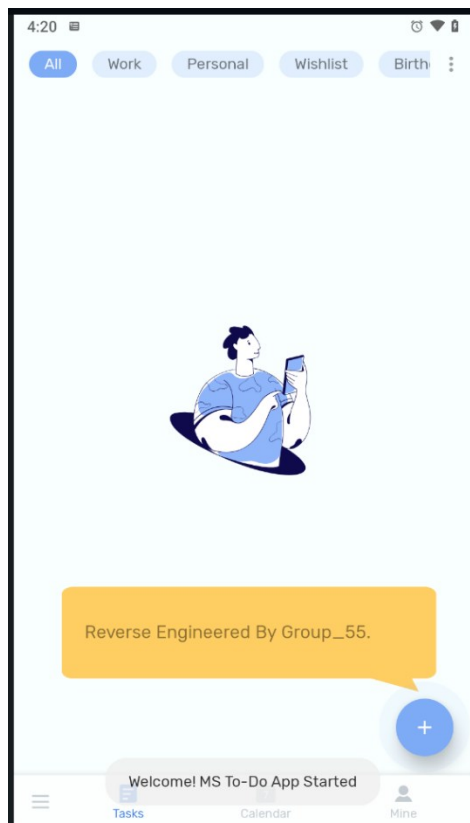
The modified application was tested using a simple checklist. The main purpose of testing was to confirm that the app installed successfully, launched without crashing, displayed the modified strings, and showed the startup message as expected.

| Test Case                  | Expected Result                                                     | Status         |
|----------------------------|---------------------------------------------------------------------|----------------|
| Install modified APK       | APK should install successfully in MEmu emulator                    | Pass           |
| Launch application         | Application should open without crash                               | Pass           |
| App label change           | Modified label should appear under app icon                         | Pass           |
| UI string change           | Modified task creation message should appear                        | Pass           |
| Startup notification/popup | Message should appear after app starts                              | Pass           |
| Basic task creation        | User should still be able to create a task                          | Pass           |
| No hidden data access      | No unauthorized data collection or network behavior should be added | To be verified |

Table 6:Testing checklist and outcomes

### 6.5 Observed Results

The expected final output of the experiment is a modified version of the To-Do List app. The app name should show the customized assignment label, the selected yellow-bar/empty-state text should display the new message, and the startup popup or notification should appear when the app is opened. The app should continue to support normal task creation and basic use after modification.



*Figure 12: Modified APK installed and running*

## 6.6 Troubleshooting Notes

- If the APK does not install, uninstall the original app first because the modified APK uses a different signature.
- If APKTool build fails, check XML syntax, missing quotes, special characters, and edited smali register counts.
- If the startup message does not appear, verify the launcher activity and the exact location where the trigger was added.
- If a notification does not appear on Android 13 or later, check notification permission behavior.
- If the app crashes after launch, use Logcat to identify the class, method, or resource causing the error.

## 7. SECURITY AND ETHICAL CONSIDERATIONS

Reverse engineering is a powerful skill in mobile security. It can help analysts understand malware, identify vulnerabilities, and improve secure development. However, it can also be misused to repackage applications, insert hidden behavior, steal information, bypass payment controls, or distribute modified apps without permission. Therefore, ethical boundaries are very important.

| Risk                       | Description                                                         | Recommended Control                                                                          |
|----------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| APK repackaging            | Attackers can modify and redistribute an app under a similar name.  | Use app signing verification, Play Integrity API, distribution controls, and user education. |
| Resource tampering         | Visible text, icons, and resources can be changed to mislead users. | Use integrity checks, server-side trust decisions, and review app distribution channels.     |
| Client-side logic weakness | Important decisions stored only in the app can be altered.          | Move critical validation to secure server-side logic.                                        |
| Excessive permissions      | Unnecessary permissions increase risk if app is modified.           | Request only minimum required permissions and explain them clearly.                          |
| Weak code protection       | Unobfuscated code is easier to analyze and modify.                  | Use obfuscation, shrinking, anti-tampering checks, and secure coding practices.              |
| Unsafe background behavior | Hidden services can be abused for persistence or data collection.   | Avoid hidden behavior and follow Android background execution restrictions.                  |

*Table 7: Security risks and recommended controls*

### 7.1 Ethical Boundaries Followed

- The APK was analyzed only for academic learning.
- Testing was carried out in an emulator, not on another person's device.
- The project did not include data theft, spying, credential harvesting, hidden persistence, or bypass of paid features.
- The startup message was visible to the user and harmless.
- The final report explains defensive lessons learned from the modification process.

## 7.2 Defensive Lessons Learned

This project shows that Android developers should not assume that packaged resources and client-side logic are impossible to modify. Important controls should not depend only on the APK. Developers should use secure server-side validation, code obfuscation, strong signing practices, integrity checks, permission minimization, and regular security testing. Users should install apps only from trusted sources and avoid unknown modified APKs.

## 8. CONCLUSION

This project successfully demonstrated a controlled Android reverse engineering and application modification workflow. The To-Do List APK was inspected, decompiled, analyzed, modified, rebuilt, signed, installed, and tested in an emulator environment.

The main practical outcomes were the modification of the app label, the modification of the visible task creation message, and the addition of a harmless startup popup or notification. These results show that APK resources and some behavior can be changed after decompile when an application is not strongly protected from tampering.

The project also highlighted important security lessons. Android developers should treat the client application as an untrusted environment and protect important decisions using server side validation and application hardening. Students should use reverse engineering knowledge responsibly and only in authorized environments.

## 9. RECOMMENDATIONS

- Use code obfuscation and shrinking to make reverse engineering more difficult.
- Use secure server-side validation for important features instead of relying only on client-side checks.
- Implement app integrity checks to detect repackaged or modified APKs.
- Request only the minimum permissions required for the application.
- Protect sensitive strings, API keys, and configuration values from direct extraction.
- Perform mobile application security testing before releasing an app.

- Educate users not to install unknown or modified APK files from untrusted sources.
- Maintain ethical rules when performing reverse engineering and use isolated lab environments.

## 10. References

- [1] Android Developers, “Android Debug Bridge (adb),” Android Developers. [Online]. Available: <https://developer.android.com/tools/adb>
- [2] Android Developers, “apksigner,” Android Developers. [Online]. Available: <https://developer.android.com/studio/command-line/apksigner>
- [3] Android Developers, “zipalign,” Android Developers. [Online]. Available: <https://developer.android.com/studio/command-line/zipalign>
- [4] Android Developers, “<uses-permission>,” Android Developers. [Online]. Available: <https://developer.android.com/guide/topics/manifest/uses-permission-element>
- [5] Apktool, “Apktool Documentation,” Apktool. [Online]. Available: <https://apktool.org/docs/>
- [6] Skylot, “jadx: Dex to Java decompiler,” GitHub. [Online]. Available: <https://github.com/skylot/jadx>
- [7] Google Play, “PDF Reader - PDF Viewer,” Google Play Store. [Online]. Available: <https://play.google.com/store/apps/details?id=pdf.reader.pdfviewer.pdfeditor>
- [8] Oracle, “Java SE Development Kit,” Oracle. [Online]. Available: <https://www.oracle.com/java/technologies/downloads/>

## 16. Appendices

### Appendix A: Key Commands Used

```
java -jar apktool.jar d "To-Do List.apk" -o ToDoList_Decompiled
java -jar apktool.jar b ToDoList_Decompiled -o ToDoList_Modified_unsigned.apk
keytool -genkey -v -keystore msapp-key.jks -keyalg RSA -keysize 2048 -validity 10000 -
alias msapp
apksigner sign --ks msapp-key.jks --out ToDoList_Modified_signed.apk
ToDoList_Modified_unsigned.apk
adb devices
adb install -r ToDoList_Modified_signed.apk
```

## Appendix B: Trigger Code Injection

```
        invoke-virtual {p0}, Lapp/todolist/activity/MainActivity;-
>finish()V

        .line 18

        .line 19

        .line 20

        return-void

        .line 21

        :cond_0

        const-string v0, "Welcome! MS To-Do App Started"
# Added for assignment: background service startup popup message

        const/4 v1, 0x1

        invoke-static {p0, v0, v1}, Landroid/widget/Toast;-
>makeText(Landroid/content/Context;Ljava/lang/CharSequence;I)Landroi
d/widget/Toast;

        move-result-object v0

        invoke-virtual {v0}, Landroid/widget/Toast;->show()V
```

## Appendix C: Concern form

### **IE3112 – Mobile Security Assignment Ethical Use & Acknowledgement Declaration Form**

---

Assignment Title - Reverse Engineering and Controlled Malware  
Simulation in Android Applications

#### **1. Purpose of the Declaration**

This form serves as a formal acknowledgement that the students participating in this assignment understand the ethical, legal, and academic responsibilities associated with performing reverse engineering and controlled malware simulation activities. The activities conducted in this assignment are strictly for educational and research purposes within the scope of the IE3112 Mobile Security course.

#### **2. Student Declaration**

We, the undersigned students, acknowledge and agree to the following conditions:

1. The reverse engineering and application modification activities performed in this assignment are for academic learning purposes only.
2. The modified Android application (APK) developed as part of this assignment will only be used in controlled environments, such as:
  - Android emulator
  - Personal testing device
  - Laboratory environment approved by the lecturer
3. We will not distribute, upload, share, or publish the modified APK or any malicious code developed during this assignment to:
  - Public repositories
  - Application stores
  - Websites
  - Social media platforms
  - Any third-party individual or organization.
4. We will not target real users, devices, organizations, or services using the modified app.
5. We will not include functionalities that perform real malicious actions, such as:
  - Data theft
  - Unauthorized credential harvesting
  - Financial fraud
  - Unauthorized SMS or call activities
  - Accessing personal user data without consent
6. All simulated malicious behavior implemented in the application will be controlled demonstrations for educational purposes only.
7. We understand that misuse of the knowledge, tools, or code developed in this assignment outside the academic scope may be illegal and may result in:
  - University disciplinary action
  - Legal consequences under applicable cybercrime laws.
8. We confirm that the selected APK application will be approved by the course lecturer before performing any reverse engineering activities.
9. We agree to comply with all ethical hacking guidelines and university academic integrity policies.

8. We confirm that the selected APK application will be approved by the course lecturer before performing any reverse engineering activities.
9. We agree to comply with all ethical hacking guidelines and university academic integrity policies.

### 3. Group Information

Course Code: IE3112

Course Title: Mobile Security

Assignment Title: Reverse Engineering and Controlled Malware Simulation

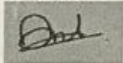
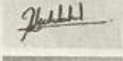
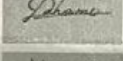
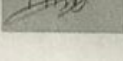
Group Number: \_\_\_\_\_ Group 55 \_\_\_\_\_

Selected Application (APK Name): PDF Reader - PDF Viewer (2.0.4)

Lecturer Approval (Initial/Signature):

*K. Thero*

### 4. Student Details and Signatures

| No | Student Name     | Student ID | Signature                                                                            | Date      |
|----|------------------|------------|--------------------------------------------------------------------------------------|-----------|
| 1  | Anulasha K.A.    | IT23139480 |    | 27.042026 |
| 2  | Abeykoon A.M.A.S | IT23403642 |   | 27.042026 |
| 3  | Anjana I.K.D     | IT23160620 |  | 27.042026 |
| 4  | Perera L.M.D     | IT23404182 |  | 27.042026 |

### 5. Lecturer Confirmation

I confirm that the selected application has been reviewed and approved for educational reverse engineering and controlled malware simulation within the scope of this course.

Lecturer Name: Tharaniyawarma. K

Signature: *K. Thero*

Date: 28/04/2026

### 6. Important Notice

Failure to comply with this declaration or misuse of the developed software outside the scope of this assignment will be treated as academic misconduct and ethical violation, and may result in disciplinary action according to university regulations.

## **Appendix D: Shared Folder**

- [MS - IE3112](#)

This folder includes :

- Technical Report (PDF)
- Modified APK file
- Source modification files
- Screen recording